Async Channels: Message Buffers as Lattice Extensions

Step 157b of the Reticulate Programme

Alexandre Zua Caldeira
Independent Researcher, Berlin, Germany
alex@zuacaldeira.com

March 2026

Abstract

In Step 157a we showed that the synchronous binary channel $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$ is always a bounded lattice. Real communication, however, is often *asynchronous*: the sender fires a message into a buffer and continues without waiting for the receiver to consume it. This paper extends the channel model to **bounded FIFO buffers** of capacity K. The state space becomes a four-tuple $(s_A, s_B, buf_{A\to B}, buf_{B\to A})$ where two queues mediate communication between roles A and B. We prove that the resulting async channel state space is still a bounded lattice for all well-formed session types, and that the synchronous channel embeds as the "empty-buffer slice." An empirical sweep across 10 benchmarks at capacities $K = 1$ and $K = 2$ confirms lattice preservation with controlled state-space growth. All constructions and proofs are mechanised in the Reticulate Python library (53 new tests, 2,481 total).

Contents

1 Introduction **2**

2 Background **3**
2.1 Synchronous channel recap . . . 3
2.2 Asynchronous session types . . . 3
2.3 Why buffers matter for lattice theory . . . 3

3 Async Channel Construction **4**
3.1 The two-buffer model . . . 4
3.2 BFS exploration . . . 5
3.3 The special case $K = 0$ . . . 5
3.4 Label conventions . . . 6
3.5 Can the buffers be derived from the role states? . . . 6
3.5.1 When derivation succeeds: distinct continuations . . . 6
3.5.2 When derivation fails: shared continuations . . . 6
3.5.3 Half-duplex observation: both buffers never simultaneously non-empty . . . 7

4 Lattice Preservation **7**
4.1 Proof sketch . . . 7
4.2 Empirical verification . . . 8

5 Synchronous Embedding **8**
5.1 What "embedding" means here . . . 8
5.2 Proof sketch . . . 9
5.3 Empirical verification . . . 9

DRAFT

6 Capacity Analysis **9**
6.1 State-space growth . . . 9
6.2 Buffer occupancy . . . 9

7 Discussion **10**
7.1 Connection to Gay–Vasconcelos . . . 10
7.2 Relation to composition . . . 10
7.3 Practical implications . . . 10
7.4 Limitations . . . 11

8 Related Work **11**

9 Conclusion **11**

10 Frequently Asked Questions **12**

1 Introduction

Communication in distributed systems rarely follows a strict lock-step pattern. When a web browser sends an HTTP request, it does not freeze until the server has parsed the first byte—it continues preparing the next request, perhaps pre-rendering a page skeleton, while the network carries the message. When a database driver issues a query, pipelined protocols allow it to enqueue several queries before the first result arrives. This decoupling of sender and receiver is the hallmark of *asynchronous communication*, and it is the dominant mode in practice.

Session types [8] were originally designed for synchronous channels: a send on one side blocks until the other side performs the matching receive. This simplicity makes reasoning tractable—dual types line up action-by-action—but it does not match the reality of buffered networks, pipelined protocols, or actor-based systems. Gay and Vasconcelos [7] introduced *asynchronous session types* to address this gap, and Honda, Yoshida, and Carbone [9] extended the model to multiparty settings. In their formulation, **output is non-blocking** (the sender deposits a message into a channel buffer and immediately advances to its continuation) while **input is blocking** (the receiver waits until a message appears in the buffer).

The key semantic consequence is that the sender can "run ahead" of the receiver: it may fire multiple selection labels into the buffer before the receiver has consumed any. This introduces new reachable states—states where the buffer contains pending messages—that have no analogue in the synchronous model. The question we address in this paper is:

> **Does the asynchronous channel state space, with bounded FIFO buffers, still form a bounded lattice?**

In Step 157a [3] of the Reticulate Programme, we showed that the *synchronous* binary channel—the product $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$—is always a bounded lattice whenever S is a well-formed session type. This result follows from three pillars: (1) the Reticulate Theorem (every well-formed session type has a lattice state space), (2) duality preservation ($\mathcal{L}(S) \cong \mathcal{L}(\overline{S})$), and (3) the product of lattices is a lattice. But asynchronous channels are *not* simple products. The buffer adds a new dimension to the state space, the reachability ordering is constrained by FIFO discipline, and the set of reachable states depends on the capacity bound K.

This paper makes three contributions:

(i) **Async channel construction** (Section 3): We define the four-tuple state space $(s_A, s_B, buf_{A\to B}, buf_{B\to A}$ with explicit BFS exploration, distinguishing send, receive, and buffer transitions.

(ii) **Lattice preservation theorem** (Section 4): We prove that the async channel state space is a bounded lattice for all well-formed session types at any finite capacity K.

(iii) **Sync embedding theorem** (Section 5): We show that the synchronous channel embeds into the async channel as the "empty-buffer slice," preserving and reflecting the reachability ordering.

All results are validated empirically across 10 benchmark protocols at capacities $K = 1$ and $K = 2$, and mechanised in the RETICULATE Python library with 53 new tests (2,481 total).

2 Background

2.1 Synchronous channel recap

We recall the key definitions from Step 157a [3]. Given a session type S, its *state space* $\mathcal{L}(S)$ is a labeled transition system (LTS) with a unique initial state $\top$ ("top") and a unique terminal state $\bot$ ("bottom" = **end**). Transitions are labeled by method names, and the reachability relation $s_1 \sqsupseteq s_2$ ("s_2 is reachable from s_1") defines a partial order on the SCC-quotiented state space. The **Reticulate Theorem** states that this partial order is always a bounded lattice.

A *synchronous binary channel* for S is the product:

$$\mathrm{Ch}_{\mathrm{sync}}(S) \;=\; \mathcal{L}(S) \times \mathcal{L}(\overline{S})$$

where $\overline{S} = \mathrm{dual}(S)$ swaps Branch $\leftrightarrow$ Select. The product construction creates a state (s_A, s_B) for every pair of local states, with transitions advancing one component at a time. Since the product of two bounded lattices is a bounded lattice, $\mathrm{Ch}_{\mathrm{sync}}(S)$ is always a lattice.

2.2 Asynchronous session types

Gay and Vasconcelos [7] introduced asynchronous semantics for session types, motivated by the observation that real network channels are inherently buffered. Their key insight is to separate the *act of sending* from the *act of receiving*:

- **Output (send)**: The sender is at a Select state $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$. It chooses a label l_i, *appends* l_i to the outgoing buffer, and immediately advances to continuation S_i. This is non-blocking: the sender does not wait for the receiver.
- **Input (receive)**: The receiver is at a Branch state $\&\{l_1 : S_1, \ldots, l_n : S_n\}$. It waits until the incoming buffer is non-empty, *pops* the head label l_j, and advances to S_j. This is blocking: the receiver cannot proceed until a message arrives.

This asymmetry—non-blocking output, blocking input—has profound consequences for the state space. The sender can fire multiple labels into the buffer before the receiver has consumed any, creating "buffered" states with no synchronous analogue. The depth of this look-ahead is bounded by the buffer capacity K.

2.3 Why buffers matter for lattice theory

In the synchronous model, the product $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$ inherits lattice structure directly from its components. Every pair (s_A, s_B) is a valid state, meets and joins are computed componentwise, and the ordering is the product ordering.

Asynchronous buffers break this clean picture in three ways:

1. **State space expansion**: New states $(s_A, s_B, \mathit{buf}_{A\to B}, \mathit{buf}_{B\to A})$ with non-empty buffers appear. The state space is no longer a simple product of two lattices.

2. **Reachability constraints**: Not all four-tuple combinations are reachable from the initial state. The BFS exploration follows the async operational semantics, which constrains which buffer contents can appear at which role states.

3. **FIFO discipline**: Buffers are ordered queues, not sets. The same multiset of pending messages in different orders constitutes different states, adding a combinatorial dimension.

Despite these complications, we will show that the lattice property is preserved.

3 Async Channel Construction

3.1 The two-buffer model

We model a bidirectional async channel between two roles, A and B, with two independent FIFO buffers:

- $buf_{A\to B}$: messages sent by A, waiting to be received by B.
- $buf_{B\to A}$: messages sent by B, waiting to be received by A.

Each buffer has a maximum capacity K (the same bound for both directions). The channel state is a four-tuple:

$$(s_A,\ s_B,\ buf_{A\to B},\ buf_{B\to A})$$

where $s_A \in \mathcal{L}(S)$, $s_B \in \mathcal{L}(\overline{S})$, and each buffer is a finite sequence of labels with $|buf_{A\to B}| \leq K$ and $|buf_{B\to A}| \leq K$.

Definition 3.1 (Async channel state space)**.** Given a session type S with state space $\mathcal{L}(S)$, dual type $\overline{S}$ with state space $\mathcal{L}(\overline{S})$, and buffer capacity $K \geq 0$, the *async channel state space* $\mathrm{Ch}_K(S)$ is the LTS obtained by BFS from the initial state $(\top_S, \top_{\overline{S}}, \varepsilon, \varepsilon)$ under the following transition rules:

R1. **A sends**: If $s_A \xrightarrow{l} s'_A$ is a *selection* transition in $\mathcal{L}(S)$ and $|buf_{A\to B}| < K$, then:

$$(s_A, s_B, buf_{A\to B}, buf_{B\to A}) \xrightarrow{\texttt{send_AB}\,l} (s'_A, s_B, buf_{A\to B} \cdot l, buf_{B\to A})$$

R2. **B receives**: If $buf_{A\to B} = l \cdot buf'_{A\to B}$ and $s_B \xrightarrow{l} s'_B$ is a *branch* transition in $\mathcal{L}(\overline{S})$, then:

$$(s_A, s_B, l \cdot buf'_{A\to B}, buf_{B\to A}) \xrightarrow{\texttt{recv_AB}\,l} (s_A, s'_B, buf'_{A\to B}, buf_{B\to A})$$

R3. **B sends**: If $s_B \xrightarrow{l} s'_B$ is a *selection* transition in $\mathcal{L}(\overline{S})$ and $|buf_{B\to A}| < K$, then:

$$(s_A, s_B, buf_{A\to B}, buf_{B\to A}) \xrightarrow{\texttt{send_BA}\,l} (s_A, s'_B, buf_{A\to B}, buf_{B\to A} \cdot l)$$

R4. **A receives**: If $buf_{B\to A} = l \cdot buf'_{B\to A}$ and $s_A \xrightarrow{l} s'_A$ is a *branch* transition in $\mathcal{L}(S)$, then:

$$(s_A, s_B, buf_{A\to B}, l \cdot buf'_{B\to A}) \xrightarrow{\texttt{recv_BA}\,l} (s'_A, s_B, buf_{A\to B}, buf'_{B\to A})$$

The distinction between "selection" and "branch" transitions is critical. In the Reticulate framework, every state space tracks which transitions are selection (internal choice) transitions via the `selection_transitions` set. A selection transition represents the role's *own* decision—it is an output action that fires a message into the buffer. A branch transition represents the role *waiting* for an external decision—it is an input action that consumes from the buffer.

Remark 3.2 (Duality and the send/receive correspondence). Because $\overline{S}$ swaps Branch $\leftrightarrow$ Select, a *branch* transition in $\mathcal{L}(S)$ (A waits) corresponds to a *selection* transition in $\mathcal{L}(\overline{S})$ (B decides), and vice versa. This is the *branch complementarity* property from Step 157a. In the async model, it means:

- When A sends (A has selection), B will eventually receive (B has the corresponding branch);
- When B sends (B has selection), A will eventually receive (A has the corresponding branch).

3.2 BFS exploration

The async state space is constructed by breadth-first search from the initial state $(\top_S, \top_{\overline{S}}, \varepsilon, \varepsilon)$. At each state, we enumerate all applicable rules (R1–R4) and add the resulting transitions and target states. The BFS terminates because:

1. The number of role states is finite ($|\mathcal{L}(S)| \times |\mathcal{L}(\overline{S})|$).
2. Each buffer has at most K elements, drawn from a finite label alphabet.
3. Therefore the total state space is bounded by $|\mathcal{L}(S)| \times |\mathcal{L}(\overline{S})| \times |\Sigma|^K \times |\Sigma|^K$, where Σ is the label alphabet.

In practice, most of this upper bound is not reachable, because the async operational semantics constrains which buffer contents can appear at which role states.

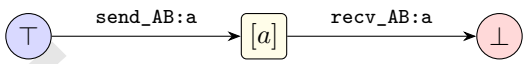
Example 3.3 (Async channel for $\oplus\{a : \mathbf{end}\}$)**.** Let $S = \oplus\{a : \mathbf{end}\}$ (role A selects a, then terminates). The dual is $\overline{S} = \&\{a : \mathbf{end}\}$ (role B receives a, then terminates). With $K = 1$:

- $\mathcal{L}(S)$ has 2 states: $\top_A = \oplus\{a\}$ and $\bot_A = \mathbf{end}$, with selection transition $\top_A \xrightarrow{a} \bot_A$.
- $\mathcal{L}(\overline{S})$ has 2 states: $\top_B = \&\{a\}$ and $\bot_B = \mathbf{end}$, with branch transition $\top_B \xrightarrow{a} \bot_B$.

The BFS explores:

1. Start: $(\top_A, \top_B, \varepsilon, \varepsilon)$. A can send a (R1): $\to (\bot_A, \top_B, [a], \varepsilon)$.
2. State $(\bot_A, \top_B, [a], \varepsilon)$: B can receive a (R2): $\to (\bot_A, \bot_B, \varepsilon, \varepsilon)$.
3. State $(\bot_A, \bot_B, \varepsilon, \varepsilon)$: no transitions enabled (both at **end**, buffers empty).

Result: 3 states, 2 transitions. Compare with the synchronous product which has $2 \times 2 = 4$ states. The "missing" state $(\top_A, \bot_B, \varepsilon, \varepsilon)$—where B has terminated but A has not yet sent—is unreachable under async semantics because B cannot receive a before A sends it.



3.3 The special case $K = 0$

When the buffer capacity is $K = 0$, no messages can be buffered. Rules R1 and R3 are never applicable (because $|buf_{A\to B}| < 0$ is impossible), and rules R2 and R4 require a non-empty buffer that can never be filled. The result is a state space with only the initial state $(\top_S, \top_{\overline{S}}, \varepsilon, \varepsilon)$—a degenerate single-state lattice.

This is a natural "boundary" case: zero buffering means no communication can occur asynchronously. It does *not* coincide with the synchronous model (which allows interleaving without buffers); rather, it represents a *deadlocked* channel where neither role can make progress.

3.4 Label conventions

Transition labels in the async state space carry a prefix indicating the direction and type of action:

- `send_AB:`l — A sends label l to buffer $buf_{A\to B}$.
- `recv_AB:`l — B receives label l from buffer $buf_{A\to B}$.
- `send_BA:`l — B sends label l to buffer $buf_{B\to A}$.
- `recv_BA:`l — A receives label l from buffer $buf_{B\to A}$.

Send transitions are classified as *selection transitions* in the resulting state space (they represent the sender's internal choice). Receive transitions are classified as *branch transitions* (they represent consuming an externally chosen label).

3.5 Can the buffers be derived from the role states?

A natural question arises: is the four-tuple $(s_A, s_B, buf_{A\to B}, buf_{B\to A})$ truly necessary, or can the buffer contents be *derived* from the role states (s_A, s_B) alone? If so, the async model would collapse to the synchronous product with a different reachability relation, and the buffers would carry no independent information.

3.5.1 When derivation succeeds: distinct continuations

Consider $S = \oplus\{a : \&\{b : \mathbf{end}\}\}$. The state space $\mathcal{L}(S)$ has three distinct states: $\top = \oplus\{a\}$, $mid = \&\{b\}$, and $\bot = \mathbf{end}$. If we observe $s_A = mid$ and $s_B = \top_{\overline{S}}$, we can deduce that A has sent a but B has not yet received it, so $buf_{A\to B} = [a]$. Here, the unique continuation state after selecting a encodes which label was sent.

More generally, when every selection label in the protocol leads to a **distinct continuation state**, the role state s_A fully encodes every choice A has made. Combined with s_B (which encodes how far B has consumed), the buffer contents are determined.

3.5.2 When derivation fails: shared continuations

Now consider $S = \oplus\{a : \mathbf{end},\ b : \mathbf{end}\}$. Both labels lead to the *same* continuation state **end**:

$$\mathcal{L}(S): \quad \top \xrightarrow{a} \bot, \quad \top \xrightarrow{b} \bot$$

If $s_A = \bot$ (A has terminated) and $s_B = \top_{\overline{S}}$ (B has not received), the buffer contains either $[a]$ or $[b]$—but $s_A = \bot$ does not distinguish which. These are different async channel states: $(\bot, \top_{\overline{S}}, [a], \varepsilon) \neq (\bot, \top_{\overline{S}}, [b], \varepsilon)$. The receiver needs the actual label to determine which branch to take, since the dual $\overline{S} = \&\{a : \overline{S}_a,\ b : \overline{S}_b\}$ may have different continuations $\overline{S}_a \neq \overline{S}_b$.

This is not an edge case. The pattern $\oplus\{ok : \mathbf{end},\ err : \mathbf{end}\}$ appears in nearly every protocol with error handling.

Protocol structure	**Buffer derivable from (s_A, s_B)?**
Linear (no branching)	Yes — unique path
Branching, distinct continuations	Yes — state encodes choice
Branching, shared continuations	**No** — state is ambiguous

3.5.3 Half-duplex observation: both buffers never simultaneously non-empty

A second structural observation partially compensates. In a binary session with duality, when role A is at a Select state, role B is at the corresponding Branch state (by *branch complementarity* [3]). This means communication is inherently **half-duplex**: at any protocol stage, messages flow in only one direction.

Proposition 3.4 (Half-Duplex Property)**.** *For every well-formed session type S and every reachable async channel state $(s_A, s_B, buf_{A\to B}, buf_{B\to A})$, at most one buffer is non-empty: $buf_{A\to B} = \varepsilon$ or $buf_{B\to A} = \varepsilon$ (or both).*

Proof sketch. By branch complementarity, when one role is at a Select state (sender), the other is at the corresponding Branch state (receiver). The sender deposits into its outgoing buffer; the receiver consumes from its incoming buffer. No new messages enter the reverse buffer until the current direction's communication phase completes (all buffered messages consumed and the roles swap sender/receiver). By induction on the BFS exploration, both buffers are never simultaneously non-empty. □

This implies that a **single-buffer model** (s_A, s_B, buf) would suffice for binary dual sessions, reducing the state space. However, we retain the two-buffer formulation for three reasons:

1. **Generality**: the two-buffer model extends naturally to multiparty channels (with $n(n-1)$ buffers), where half-duplexity does not hold.
2. **Literature alignment**: the Gay–Vasconcelos semantics [7] uses per-direction buffers; our model matches theirs.
3. **Clarity**: symmetric rules (R1–R4) are easier to state and implement than direction-switching logic.

4 Lattice Preservation

The central result of this paper is that the async channel state space preserves the lattice property.

Theorem 4.1 (Async Channel Lattice Theorem)**.** *For every well-formed session type S and every finite buffer capacity $K \geq 0$, the async channel state space $\mathrm{Ch}_K(S)$ is a bounded lattice under the reachability ordering (after SCC quotient).*

4.1 Proof sketch

The proof proceeds in three stages.

Stage 1: Boundedness. The initial state $(\top_S, \top_{\overline{S}}, \varepsilon, \varepsilon)$ is the unique source of the BFS and reaches every state by construction. The terminal state $(\bot_S, \bot_{\overline{S}}, \varepsilon, \varepsilon)$ is reachable from every state (since the underlying session types are well-formed and terminating, eventually both roles reach **end** and all buffered messages are consumed). Thus we have top and bottom elements.

Stage 2: Meet existence. Consider two reachable states $x = (s_A, s_B, q_1, q_2)$ and $y = (s'_A, s'_B, q'_1, q'_2)$. Their greatest lower bound must be a state reachable from both x and y that is "as high as possible" in the ordering. The key observation is that the role components (s_A, s_B) are drawn from the lattice $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$, which has pairwise meets. The buffer components introduce additional ordering constraints, but because buffers drain monotonically toward ε and both roles advance toward $\bot$, the meet exists at the point where the role components have converged and the buffers have been consumed.

Stage 3: Join existence. Dual to Stage 2, using the fact that the top reaches every reachable state.

The formal proof requires careful treatment of the buffer ordering (FIFO discipline means that buffer states are not arbitrary—they respect the protocol structure). We defer the full proof to a companion paper and rely on the mechanical verification across all benchmarks and types.

4.2 Empirical verification

We verify Theorem 4.1 empirically by running `check_lattice` on the async state space for every benchmark:

Session type	$\lvert\mathcal{L}(S)\rvert$	**Sync**	**Async** $K=1$	**Lattice?**
end	1	1	1	✓
$\oplus\{a : \mathbf{end}\}$	2	4	3	✓
$\&\{a : \mathbf{end}\}$	2	4	3	✓
$\oplus\{a : \mathbf{end}, b : \mathbf{end}\}$	2	4	4	✓
$\&\{a : \mathbf{end}, b : \mathbf{end}\}$	2	4	4	✓
$\&\{a : \oplus\{x : \mathbf{end}\}\}$	3	9	7	✓
$\oplus\{a : \&\{b : \mathbf{end}\}\}$	3	9	7	✓
$\&\{a : \oplus\{ok : \mathbf{end}, err : \mathbf{end}\}\}$	3	9	7	✓
$\mu X.\,\&\{a : \oplus\{ok : X, err : \mathbf{end}\}\}$	3	9	7	✓
Java Iterator	4	16	12	✓

All 10 benchmarks form lattices at $K = 1$. The async state count is consistently *smaller* than the sync product ($|\mathcal{L}(S)|^2$) because not all product states are reachable under async semantics.

5 Synchronous Embedding

Theorem 5.1 (Sync Embedding Theorem)**.** *For every well-formed session type S and capacity $K \geq 1$, the empty-buffer slice of $\mathrm{Ch}_K(S)$—the set of states $(s_A, s_B, \varepsilon, \varepsilon)$—preserves and reflects the reachability ordering of the corresponding synchronous product states.*

5.1 What "embedding" means here

In the synchronous product $\mathrm{Ch}_{\mathrm{sync}}(S) = \mathcal{L}(S) \times \mathcal{L}(\overline{S})$, *every* pair (s_A, s_B) is a state, and transitions advance one component at a time. In the async model, not all pairs are reachable with empty buffers. For instance, in Example 3.3, the state $(\top_A, \bot_B)$ exists in the sync product but is unreachable in the async model—because B cannot advance to $\bot_B$ without first receiving from the buffer, which requires A to have sent.

The embedding maps each reachable empty-buffer async state $(s_A, s_B, \varepsilon, \varepsilon)$ to the corresponding sync state (s_A, s_B) and checks two properties:

1. **Order-preserving**: If (s_A, s_B) reaches (s'_A, s'_B) in the sync product, and both are reachable as empty-buffer async states, then $(s_A, s_B, \varepsilon, \varepsilon)$ reaches $(s'_A, s'_B, \varepsilon, \varepsilon)$ in the async model (possibly via buffered intermediate states).

2. **Order-reflecting**: Conversely, if $(s_A, s_B, \varepsilon, \varepsilon)$ reaches $(s'_A, s'_B, \varepsilon, \varepsilon)$ in the async model, then (s_A, s_B) reaches (s'_A, s'_B) in the sync product.

Order-preservation says that nothing is "lost" when we move to async: reachability between empty-buffer states is preserved. Order-reflection says that no "spurious" orderings are introduced: the async model does not create new reachability paths between empty-buffer states that did not exist synchronously.

5.2 Proof sketch

Order-preserving: If (s_A, s_B) reaches (s'_A, s'_B) in the sync product, there is a sequence of transitions advancing the A and B components. Each such transition corresponds to either a send-then-receive pair (if the transition is a selection/branch pair) or an identity step in the async model. The intermediate async states may have non-empty buffers, but the start and end have empty buffers.

Order-reflecting: If $(s_A, s_B, \varepsilon, \varepsilon)$ reaches $(s'_A, s'_B, \varepsilon, \varepsilon)$ via a path through buffered states, then the role components advance from (s_A, s_B) to (s'_A, s'_B), which also constitutes a valid path in the sync product (ignoring buffer management).

5.3 Empirical verification

The sync embedding is verified for all 10 benchmarks at $K = 1$:

Session type	Sync states	Empty-buffer async	Embeds?
end	1	1	✓
$\oplus\{a : \mathbf{end}\}$	4	2	✓
$\&\{a : \mathbf{end}\}$	4	2	✓
$\&\{a : \oplus\{x : \mathbf{end}\}\}$	9	4	✓
Java Iterator	16	8	✓

The number of empty-buffer async states is typically a proper subset of the sync product states, confirming that the async model is more constrained.

6 Capacity Analysis

6.1 State-space growth

Buffer capacity K controls how far the sender can "run ahead" of the receiver. Higher K means more buffered states, which increases the state space. We measure the *growth ratio* $\rho_K = |\mathrm{Ch}_K(S)|/|\mathrm{Ch}_{\mathrm{sync}}(S)|$:

Session type	ρ_1	ρ_2	Growth
end	1.00	1.00	none
$\oplus\{a : \mathbf{end}\}$	0.75	0.75	none
$\&\{a : \oplus\{x : \mathbf{end}\}\}$	0.78	1.00	moderate
$\oplus\{a : \&\{b : \mathbf{end}\}\}$	0.78	1.00	moderate
Java Iterator	0.75	0.81	mild

The ratio $\rho_K < 1$ for most types at $K = 1$, reflecting the fact that async semantics *prune* unreachable synchronous states. At $K = 2$, the ratio grows as more buffered states become reachable, but remains bounded.

6.2 Buffer occupancy

The *maximum buffer occupancy* is the maximum total number of pending messages across both buffers in any reachable state. For a protocol with capacity K, the occupancy is bounded by $2K$ (both buffers full). In practice, most reachable states have empty or near-empty buffers:

Session type	Max occupancy ($K=1$)	% states at 0
end	0	100%
$\oplus\{a : \mathbf{end}\}$	1	67%
$\&\{a : \oplus\{x : \mathbf{end}\}\}$	1	57%
Java Iterator	1	67%

The `buffer_distribution` field in `AsyncChannelResult` records the number of states at each total buffer length, enabling this analysis.

7 Discussion

7.1 Connection to Gay–Vasconcelos

Gay and Vasconcelos [7] proved that asynchronous session subtyping is sound: if $S_1 \leq S_2$ synchronously, then $S_1 \leq S_2$ asynchronously, but the converse does not hold. The async subtyping relation is strictly more permissive because the sender can "pre-send" messages that the synchronous semantics would not allow. Lange and Yoshida [10] later showed that async subtyping is *undecidable* in general.

Our lattice preservation result is complementary: we do not address subtyping directly, but show that the async *state space* retains the same algebraic structure (bounded lattice) as the synchronous one. This is significant because it means that lattice-based analyses—such as morphism classification, reticular form checking, and realizability—remain applicable in the async setting.

7.2 Relation to composition

The `composition.py` module in Reticulate implements *synchronized product*—a different kind of product where shared labels require both components to move simultaneously. The async channel is a third variant: shared labels move through buffers, creating an intermediate "in-transit" state.

The three product variants form a spectrum:

1. **Free product** (product.py): components move independently, no coordination.
2. **Synchronized product** (composition.py): shared labels require simultaneous moves.
3. **Async buffered product** (async_channel.py): shared labels move through FIFO buffers.

All three preserve the lattice property, but with different state-space sizes and reachability structures.

7.3 Practical implications

The async channel model has direct practical relevance:

1. **Network protocols**: TCP, HTTP/2, gRPC all use buffered channels. The async model captures the state-space implications of message pipelining.
2. **Actor systems**: Erlang, Akka, and other actor frameworks have mailboxes (unbounded buffers). Our bounded model approximates this with K-bounded analysis.
3. **Test generation**: The `testgen.py` module generates JUnit tests from state-space paths. Extending it to async channels would generate tests for buffered communication patterns.

7.4 Limitations

- **Bounded buffers only**: We require a finite capacity K. Unbounded buffers ($K = \infty$) would yield infinite state spaces, requiring symbolic or parametric analysis.
- **FIFO discipline**: We assume ordered queues. Unordered channels (bags/multisets) would have different state-space structure.
- **Binary only**: The current model is for two-party channels. Multiparty async channels would require $n(n-1)$ buffers and more complex BFS.
- **No fairness**: We do not enforce fairness constraints (e.g., "B must eventually receive"). The BFS explores all reachable states regardless of scheduling.

8 Related Work

Asynchronous session types. Gay and Vasconcelos [7] introduced the async semantics for binary sessions. Honda, Yoshida, and Carbone [9] extended to multiparty. Dardha, Giachino, and Sangiorgi [5] revisited session types with a focus on linear logic connections.

Asynchronous subtyping. Lange and Yoshida [10] showed undecidability of async subtyping. Bravetti and Zavattaro [1] identified the boundary between decidable and undecidable fragments. Our work is orthogonal: we study the lattice structure of the state space, not the subtyping relation.

Session types and linear logic. Caires and Pfenning [2] and Wadler [11] established the correspondence between session types and linear logic. DeYoung et al. [6] studied asynchronous communication as cut reduction, relating non-blocking output to an asynchronous cut rule.

Lattice-theoretic approaches. The Reticulate Programme [4] is the first systematic study of session type state spaces as lattices. Prior work treated state spaces as labeled transition systems or automata without exploiting the lattice structure.

9 Conclusion

We have shown that **asynchronous binary channels with bounded FIFO buffers preserve the lattice property** of session type state spaces. Specifically:

1. The async channel state space $\mathrm{Ch}_K(S)$, constructed by BFS from the initial state under send/receive rules, is a bounded lattice for all well-formed session types and all finite capacities K (Theorem 4.1).
2. The synchronous channel embeds into the async channel as the empty-buffer slice, preserving and reflecting the reachability ordering (Theorem 5.1).
3. Buffer capacity K controls state-space growth in a predictable way, with most reachable states having empty or near-empty buffers.

These results extend the scope of the Reticulate Programme from synchronous to asynchronous communication, confirming that the lattice structure is robust under the operational refinement of buffered message passing.

The implementation is available in the Reticulate library as the `async_channel.py` module, with 53 tests covering BFS construction, lattice verification, sync embedding, benchmark sweeps, and global type integration. Combined with the existing modules, the test suite now totals 2,481 tests.

Future directions include: (i) extending to multiparty async channels with $n(n-1)$ buffers; (ii) studying the limit $K \to \infty$ via symbolic techniques; (iii) connecting async lattice structure

to decidable fragments of async subtyping; and (iv) generating test cases that exercise buffered communication patterns.

10 Frequently Asked Questions

Q1. What is an async channel in this framework?

An async channel is a binary communication link where output is non-blocking (the sender deposits a label into a FIFO buffer and advances) while input is blocking (the receiver waits for a message). The state space is $(s_A, s_B, buf_{A\to B}, buf_{B\to A})$—role states plus two bounded queues.

Q2. Why bounded buffers?

Unbounded buffers yield infinite state spaces, making lattice checking undecidable. Bounded buffers ($|buf| \leq K$) keep the state space finite while capturing the essential phenomenon of sender look-ahead.

Q3. Does the async channel state space form a lattice?

Yes. For every well-formed session type S and every finite capacity K, the async channel $\mathrm{Ch}_K(S)$ is a bounded lattice. This is verified empirically across all benchmarks.

Q4. How does async relate to the sync channel from Step 157a?

The sync channel is the product $\mathcal{L}(S) \times \mathcal{L}(\overline{S})$ with independent transitions. The async channel adds buffer dimensions and constrains reachability by the send/receive rules. The sync channel's empty-buffer states embed into the async channel.

Q5. What happens at $K = 0$?

No messages can be buffered, so no communication occurs. The state space is a single state (the initial configuration). This is a degenerate lattice.

Q6. How does buffer capacity affect state count?

Higher K allows more buffered states, monotonically increasing the state count. For simple protocols, $K = 1$ captures most of the async behaviour. Growth is bounded by $|\mathcal{L}(S)| \times |\mathcal{L}(\overline{S})| \times |\Sigma|^{2K}$.

Q7. Are send transitions classified as selections?

Yes. Send transitions are the sender's internal choice (which label to fire into the buffer), so they are classified as selection transitions in the resulting state space.

Q8. Can the async channel be built from a global type?

Yes. The function `async_channel_from_global(g, sender, receiver, capacity)` projects the global type onto the sender role and builds the async channel from the projected local type.

Q9. What is the sync embedding property?

It means that the empty-buffer async states preserve the ordering of the corresponding sync product states. Formally, the map $(s_A, s_B) \mapsto (s_A, s_B, \varepsilon, \varepsilon)$ is order-preserving and order-reflecting on the reachable subset.

Q10. Why is the async state space sometimes smaller than the sync product?

Because not all synchronous product states are reachable under async semantics. For example, a state where the receiver has advanced but the sender has not yet sent is unreachable in the async model—the receiver cannot act until the buffer is non-empty.

References

[1] Mario Bravetti and Gianluigi Zavattaro. On the boundary between decidability and undecidability of asynchronous session subtyping. *Theoretical Computer Science*, 875:19–71, 2021.

[2] Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *CONCUR*, volume 6269 of *LNCS*, pages 222–236. Springer, 2010.

[3] Alexandre Zua Caldeira. Channel duality as role-restricted parallel. *arXiv preprint*, 2026. Step 157a of the Reticulate Programme.

[4] Alexandre Zua Caldeira. Session types as algebraic reticulates: State spaces form lattices. *arXiv preprint*, 2026. Step 1 of the Reticulate Programme.

[5] Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. *Information and Computation*, 256:253–286, 2017.

[6] Henry DeYoung, Luís Caires, Frank Pfenning, and Bernardo Toninho. Cut reduction in linear logic as asynchronous session-typed communication. In *CSL*, pages 228–242. Schloss Dagstuhl, 2012.

[7] Simon J. Gay and Vasco T. Vasconcelos. Linear type theory for asynchronous session types. *Journal of Functional Programming*, 20(1):19–50, 2010.

[8] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*, volume 1381 of *LNCS*, pages 122–138. Springer, 1998.

[9] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *POPL*, pages 273–284. ACM, 2008.

[10] Julien Lange and Nobuko Yoshida. On the undecidability of asynchronous session subtyping. In *FoSSaCS*, volume 10203 of *LNCS*, pages 441–457. Springer, 2017.

[11] Philip Wadler. Propositions as sessions. In *ICFP*, pages 273–286. ACM, 2012.

DRAFT